

Unitary Stabilisation

Definition: Define the *PIC* as a realisation of the 4×4 unitary operation U , where:

$$y = U(H)x$$

Where:

- $x \in \mathbb{C}^4$ is the optical input to the PIC
- $y \in \mathbb{C}^4$ is the optical output of the PIC
- $U(H) \in \mathbb{C}^{4 \times 4}$ is the transformation implemented by the PIC,
- H : heater-value vector controlling the PIC.

The PIC operation is thus represented as: $P(x, H) = y$.

Problem Statement: Train a model to learn the relationship between x , H and y , such that $f_\theta(y) \rightarrow (x, H)$. The PIC should then satisfy $P(H, x) \approx y$. Mathematically, we have:

$$(\hat{x}, \hat{H}) = f_\theta(y_{\text{target}})$$

and,

$$\hat{y} = P(\hat{H}, \hat{x})$$

So that the loss to be minimized during training is:

$$\mathcal{L} = \left\| P(\hat{H}, \hat{x}) - y_{\text{target}} \right\|^2.$$

Appendix: Parameter-efficient transfer learning

The original large model learns $f_\theta(y) \rightarrow (x, H)$. We train only a small subset of weights, and the remaining (ones that do not change beyond a threshold during training) are frozen. This results in a much smaller model that can be trained faster, and updated iteratively based on outputs of the PIC.

Deliverables: To reproduce the experiment, we would require:

- Training dataset
- Model, along with weights
- Training/Evaluation scripts

Via: GitHub(preferred for the scripts), or Google Drive/OneDrive(for the model weights/datasets).